

Exploring the latest ISA ARMv9

Complex Engineering Problem

Prepared By:

Maria Kousar	SE-24004
Raazia	SE-24009
Syed Rafiullah	SE-24040
Tahira Shafeeq	SE-24301
Ali Raza	SE-24302

DEPARTMENT OF SOFTWARE ENGINEERING
BACHELORS IN SOFTWARE ENGINEERING
Course Code: SE-209
Course Title: Computer Architecture & Logic Design
Complex Engineering Problem
SE Batch 2024, Fall Semester 2025

CLO Covered:

CLO 3: (C3-PLO3)

Complex problem-solving attributes (CPA) covered (as per PEC - OBA manual – 2019)

CPA-1 Depth of analysis required: Have no obvious solution and require abstract thinking, originality in analysis to formulate suitable models.

CPA-2 Level of interaction: Require resolution of significant problems arising from interactions between wideranging or conflicting technical, engineering or other issues.

CPA-3 Familiarity: Can extend beyond previous experiences by applying principles-based approaches.

Complex Engineering Problem (CEP): *Exploring the Latest Instruction Set Architecture (ISA)*

Problem Statement

With the rapid advancements in computer architecture, new Instruction Set Architectures (ISAs) are continually developed to improve processing speed, power efficiency, and computational capabilities. This complex engineering problem (CEP) requires you to study a modern ISA (e.g., RISC-V, ARMv9, IBM Power10, etc.) and submit a detailed report analyzing its architecture, unique features, benefits, and limitations compared to previous ISAs.

Project Guidelines

1. ISA Selection:

- Choose one of the latest ISAs that has been introduced or significantly updated in the last 5-10 years. Recommended options include:
 - **RISC-V:** An open-source ISA with modular extensions.
 - **ARMv9:** The latest ARM architecture with a focus on performance and AI.
 - **IBM Power10:** High-performance architecture optimized for data-intensive tasks.
 - **Other recent ISAs:** With approval from your instructor.

2. Report Structure:

- **Introduction:** Briefly describe the chosen ISA and its history.
- **Technical Specifications:**
 - Key features (e.g., instruction set types, register file, memory hierarchy)

- Architectural design (e.g., pipelining)
- Power and performance optimizations.
- **Comparison with Prior Architectures:**
 - List at least two preceding ISAs and compare features, design philosophies, and enhancements.
 - Highlight the advancements and trade-offs.
- **Application Domains:**
 - Discuss the primary use cases, applications, and target industries for the ISA.
 - Explore unique advantages of this ISA for specific workloads (e.g., AI, data analytics, gaming).
- **Future Prospects:** Predictions on how this ISA could evolve, with insights on possible improvements.

Deliverables

1. **Written Report** (Max 3000 words): Due by Week 14.
 - The report should be well-organized, with headings, subheadings, and clear formatting.
 - Include diagrams and tables where appropriate to illustrate ISA components.
 - References should be properly cited in IEEE format.

Scoring Guide

- **Total Marks: 10**
 - Each criterion is worth **2 marks**, rated as follows:
 - **0 = Unsatisfactory**
 - **1 = Basic**
 - **2 = Excellent**

DEPARTMENT OF SOFTWARE ENGINEERING
BACHELORS IN SOFTWARE ENGINEERING
Course Code: SE-209
Course Title: Computer Architecture & Logic Design
Complex Engineering Problem
SE Batch 2024, Fall Semester 2025

Name and Roll Number	Maria Kousar	SE-24004
	Raazia	SE-24009
	Syed Rafiullah	SE-24040
	Tahira Shafeeq	SE-24301
	Ali Raza	SE-24302

Assessment Rubric

Criteria	CPA Mapping	0 (Unsatisfactory)	1 (Basic)	2 (Excellent)
Depth of ISA Analysis	CPA-1: Depth of analysis	Superficial or minimal analysis of ISA; lacks technical details	Basic understanding with some technical insights; limited detail in ISA components	Thorough, detailed technical analysis covering all ISA features
Comparative Insight with Prior ISAs	CPA-1: Depth of analysis	No comparison or irrelevant details	Basic comparison with limited insight into ISA advancements	Clear, insightful comparison highlighting advancements and trade-offs
Application and Relevance	CPA-2: Level of interaction	No mention of real-world applications or relevance to ISA usage	Some relevance but limited connection to real-world applications	Well-connected to relevant applications, clear use case relevance
Future Prospects	CPA-3: Familiarity	Lacks any outlook or predictions	Basic future outlook with limited analysis	Thoughtful, well-supported predictions for ISA development
Report Quality	CPA-3: Familiarity	Disorganized, lacks clarity and structure	Basic organization, but lacks clarity or structure in parts	Highly organized, clear, and engaging presentation with good structure
Total				
Signature of Teacher				

Table of Contents

1. INTRODUCTION.....	4
1.1. What is an ISA?	4
For instance:	4
1.2. Types of ISA.....	4
1.2.1. RISC (Reduced Instruction Set Computer)	4
1.2.2. CISC (Complex Instruction Set Computer)	5
1.3. Brief history of ARM. [3].....	5
1.3.1. From Acorn to Global Leader	5
1.3.2. The Beginning: Acorn Computers and a Big Problem	5
1.3.3. The Brilliant Idea: Building Their Own Chip	5
1.3.4. The First Success	5
1.3.5. Forming a New Company	6
1.3.6. The Result: Dominance in a Connected World	6
1.4. Why Armv9? [1][2].....	7
1.4.1. The Need for Smarter Performance	7
1.4.2. The Demand for Pervasive AI	7
1.4.3. The Critical Need for Enhanced Security	8
1.5. When and which company developed Armv9?	8
1.6. Conclusion	8
2. Technical Specifications of Armv9	9
2.1. Type: RISC (Reduced Instruction Set Computing)	10
2.2. Instruction Set Features	10
2.2.1. AArch64: The Foundation of Armv9	10
2.2.2. Load/Store Architecture.....	11
2.2.3. SIMD and NEON Extensions	11
2.2.4. SVE (Scalable Vector Extension)	12
2.2.5. SVE2 (Scalable Vector Extension 2)	12
Applications of SVE2:	13
2.2.6. SME (Scalable Matrix Extension)	13
2.2.7. Security-Related Instruction Extensions	14
Major Security Instruction Extensions:.....	14

2.2.8. Floating-Point and Advanced Arithmetic Instructions	14
2.3. Register File (Armv9)	15
2.3.1 The AArch64 Registers of an Apple Silicon Macs (M1):	18
Figure [5] AArch64 Registers of an Apple Silicon Macs.....	18
2.4. Memory Hierarchy	18
2.4.1. CPU Registers	19
2.4.2. Cache Memory	19
2.4.3. Main Memory (RAM)	19
2.4.4. Secondary Storage	19
2.4.5. Speed vs. Capacity	19
2.4.6. Data Location	19
2.4.7. Cache Management	20
2.5. Pipelining and Parallel Execution	20
2.5.1. Five-Stage Pipelining (found in ARM9 and a foundation for later cores)	20
2.5.2. Parallel execution in Armv9.....	21
2.6. Power and Performance Optimizations	22
2.6.1. Performance optimizations	22
2.6.2. Power optimizations	23
2.7. Security Features (Confidential Compute Architecture)	23
2.7.1. Confidential Compute Architecture (CCA)	23
2.7.2. Core components.....	24
3. Armv9 Comparisons: Overview and Key Features	25
3.1. Comparison with Previous Architectures	25
3.2. Feature Comparison.....	25
3.2.1. AI / ML and Vector Extensions	25
3.2.2. Performance Comparison of x86, ARM, and RISC-Processors	26
3.2.3. Security Features	28
3.2.4. Instruction Set / ISA Philosophy	28
3.2.5. Cost and Availability	29
3.3. Real Life Comparison.....	29
3.3.1. x86	29
3.3.2. ARM	29
3.4. Summary Table	30
3.4.1. ARMv8 vs Armv9	30

3.4.2. x86 vs ARM	30
4. Application Domains	31
4.1. Mobile Computing (Smartphones)	31
4.2. Laptop and personal computing.....	31
4.3. Data Centers and Cloud Computing	31
4.4. AI and Machine Learning.....	32
4.5. Automotive and Advanced Driver-Assistance Systems (ADAS).....	32
4.7. Conclusion	33
5. Future Prospects	33
5.1. Pervasive and Specialized AI from Cloud to Edge	33
5.1.1. Smarter Edge Devices.....	33
5.1.2. Ubiquitous AI in Consumer Tech:	34
5.2. Redefining Security with Confidential Compute as Standard	34
5.2.1. Industry-Wide Adoption.....	34
5.2.2. Expanding the Security Perimeter	34
5.2.3. Eliminating Memory Safety Vulnerabilities	34
5.3. Challenging and Replacing Incumbent Architectures	35
5.3.1. The Cloud and Datacenter.....	35
5.3.2. The Client PC Market.....	35
5.3.3. The Total Compute Design Approach for Arm	36
5.4 Conclusion	37
6. Conclusion	37
7. XOR Rotatory Password Hashing Algorithm (ARMv9 Assembly)	38
7.1. Introduction	38
7.2. Technical Implementation.....	38
7.3. Code Explanation	43
7.3.1. Algorithm	43
7.4. Results	44
7.4.1. Simple Text	44
7.4.2. Mixed Case & Numbers.....	44
7.4.3. Special Characters	45
7.4.4. Longer Inputs	45
7.4.5. Edge Cases	45
8. References:	46

1. INTRODUCTION

1.1. What is an ISA?

An Instruction Set Architecture (ISA) is a part of the abstract model of a computer that defines how a processor is going to understand and execute instructions given by software. In other words, “The ISA defines what the CPU can do, serving as the fundamental set of rules for how the processor operates”

The ISA acts as an interface between the hardware and the software.

The ISA provides the only way through which a program(software) is able to interact with the hardware. It can be viewed as a programmer’s manual because it’s the portion of the machine that is visible to the assembly language programmer, the compiler writer, and the application programmer. Think of it as a language that hardware speaks.

For instance:

When we write a program in high level language, we are basically giving instructions to computer to perform a certain task. Now computer can’t understand that language so we need a compiler to convert it in machine code, a series of binary instructions. These instructions must follow the rules of CPU’s ISA like ARM, RISC-V, x86 etc. Therefore, software compiled for one ISA will not run on a processor with a different ISA without emulation or recompilation.

1.2. Types of ISA

There are two types of Instruction set architectures.

1. RISC (reduced instruction set computing).
2. CISC (complex instruction set computing).

1.2.1. RISC (Reduced Instruction Set Computer)

It uses a small, simple set of basic instructions and designed to execute instruction in a single clock cycle. This simple approach makes RISC chips very power efficient. Perfect for phones and tablets where battery life matters.

Examples: RISC-V, ARM, MIPS etc.

1.2.2. CISC (Complex Instruction Set Computer)

It uses a large, feature-rich set of instructions and designed to execute instruction in multiple cycles. These sophisticated instructions can do more work in single steps. Well suited for demanding computer workloads.

Examples: Intel x86, Motorola 64k family, VAX (Virtual Address eXtension) etc.

1.3. Brief history of ARM. [3]

1.3.1. From Acorn to Global Leader

The story of ARM is a journey from a small project to a technology that powers the modern world.

1.3.2. The Beginning: Acorn Computers and a Big Problem

In the early 1980s, a British company named Acorn Computers was successful with its BBC Microcomputer. But after observing that new computers from IBM and Apple are using more powerful processors. Acorn tested these existing processors and found them to be too slow, expensive, or inefficient for their ambitious upcoming generation computers.

1.3.3. The Brilliant Idea: Building Their Own Chip

Now they decided to have a permanent solution of this inefficiency. Acorn's engineers, Sophie Wilson and Steve Furber, decided to design their own CPU on efficient processor design (called RISC). Their aim was to create a simple, fast, and power-efficient processor. This project was called the Acorn RISC Machine/ARM in short.

1.3.4. The First Success

The first ARM prototype ARM1 worked perfectly in first its test in 1985 and was made by using only about 30,000 transistors (much fewer than other chips) yet very efficient. This chip played a crucial role in order to develop the Acorn's next big product.

The ARM2 processor was launched in 1986 and refined in 1987, which was the first commercial implementation of the ARM architecture. Its key

innovations were a hardware multiplier for faster calculations and a Fast Interrupt (FIQ) system for rapid response to events. This high level performance was achieved with a remarkably simple design containing only 30,000 transistors, which also resulted in low power consumption.

The ARM2's success was a big support to the ARM3, which added a 4KB cache ,later ARM6 generation finally extended a full 32 bits address bus,completing the evolution from the original 26-bit design and maintaining the stage for the modern ARM processor.

1.3.5. Forming a New Company

The real comeback happened when Apple Computers needed a powerful but extremely efficient processor for its new "Newton" PDA. Apple partnered with Acorn and VLSI Technology. In 1990, they spun off the ARM design team into a new, independent company called "Advanced RISC Machines Ltd" later became Arm Holdings.

Now a master-mind idea executed and instead of making its own computers, the new Arm company would focus on one thing, designing the blueprints for processors and licensing those designs to other companies.

1.3.6. The Result: Dominance in a Connected World

This licensing model was perfectly worked with the increment of the mobile phone industry. Companies like Nokia needed chips that must be small and didn't drain the battery. ARM's designs were the perfect fit for it. Over the years, ARM's architecture evolved, they added critical features like 64-bit support. Today, over 230 billion ARM-based chips have been shipped, making it the most widely used processor architecture on the globe, found in virtually every smartphone, tablet, and countless other devices.

1.4. Why Armv9? [1][2]

Armv9-A is the latest generation of the Arm A-profile CPU architecture, designed to meet the compute, efficiency, and security demands of AI-

driven systems. It introduces Scalable Vector Extension 2 (SVE2) and Scalable Matrix Extension (SME) for higher-throughput data processing, and Realm Management Extension (RME) for confidential computing. These features enable developers and SoC architects to build scalable, secure, and power-efficient platforms across edge and cloud deployments.[1]

Armv9 is the first new architecture in a decade, designed not just to be faster, but to be smarter and more secure for today's specific challenges.

1.4.1. The Need for Smarter Performance

- **The Challenge:** Modern applications like 5G, virtual reality, scientific computing process, immense data streams, swamped the traditional methods for improving speed.
- **Solution:** **SVE2** & **SME**
Scalable Vector Extension 2 (SVE2) and Scalable Matrix Extension (SME) upgraded the processor's ability to handle data by working on large, flexible streams simultaneously , rather than one piece at a time. This gives an efficiency boost for:
 - **Artificial Intelligence (AI) and Machine Learning (ML):** The core mathematics of AI relies on the data processing with parallelism.
 - **Multimedia and Signal Processing:** Tasks like image or voice recognition and 5G data encoding become much faster and more power-efficient with the commencement of Armv9.

1.4.2. The Demand for Pervasive AI

- **The Challenge:** AI is no longer just in the cloud, it's in our pockets, cars, and homes even it became a crucial part for our daily life which requires processors that can handle AI workloads locally for speed and privacy.
- **Solution:**
By integrating SVE2 directly into the CPU, Armv9 is built to accelerate on-device AI feature. This means your phone can now easily run advanced language translation, photo editing, and health monitoring features fastly and efficiently without needing a constant internet connection, which also keeps the security of your personal data.

1.4.3. The Critical Need for Enhanced Security

- **The Challenge:** when we store and process more sensitive data in the cloud and on our devices too, we need hardware-level security that software alone cannot provide.
- **Solution: Confidential Compute Architecture (CCA)**

Armv9 CCA created an areas inside the processor called Realms. Realms act like digital vaults where sensitive code and data can be processed. The shocking point is that this data is protected from *everything*, including the main operating system. This is an important part for:

 - **Protecting personal data** on your device.
 - **Securing business information** especially in shared cloud servers.
 - **Building a new standard of trust** in computing.

In a nut shell, Armv9 was created for speed improvements within the premises of simplicity. It provides the specialized tools needed for a world driven by data, AI, and the need for absolute security, making it almost 100% sure that it will power the next decade of innovation.

1.5. When and which company developed Armv9?

ARM of version Armv9 was released in March ,2021. It was developed by ARM Ltd (ARM holdings), a company based in Cambridge, UK.

1.6. Conclusion

In short,Armv9 has an incredible capabilities in vector processing , matrix processing , confidential computing , power efficiency , security and AI features. So, it is best suited strategic evolution from general-purpose computing to specialized, with in a secure processing in this AI era.

2. Technical Specifications of Armv9

In this section, the focus is on the technical specifications of the Armv9 architecture, providing an overview of its core design principles and technological advancements. The section outlines the architectural elements that define Armv9's performance, efficiency, and security, offering

insights into how these specifications contribute to its overall functionality and innovation.

To illustrate the internal organization of the processor, the block diagram shown below presents the structural arrangement of the major components within the Armv9 architecture. It highlights the interaction between key functional units such as the instruction pipeline, instruction decode and control logic, register bank, arithmetic logic unit (ALU), shifter, multiplier, and data paths. The diagram shows how instructions are fetched, decoded, and executed through a coordinated flow of data and control signals. Components like multiplexers, sign extension units, and memory interfaces manage data movement and synchronization across the system, ensuring smooth and efficient execution. Overall, the block diagram reflects the modular and streamlined design of the Armv9 architecture, emphasizing its balance between high performance, scalability, and power efficiency.

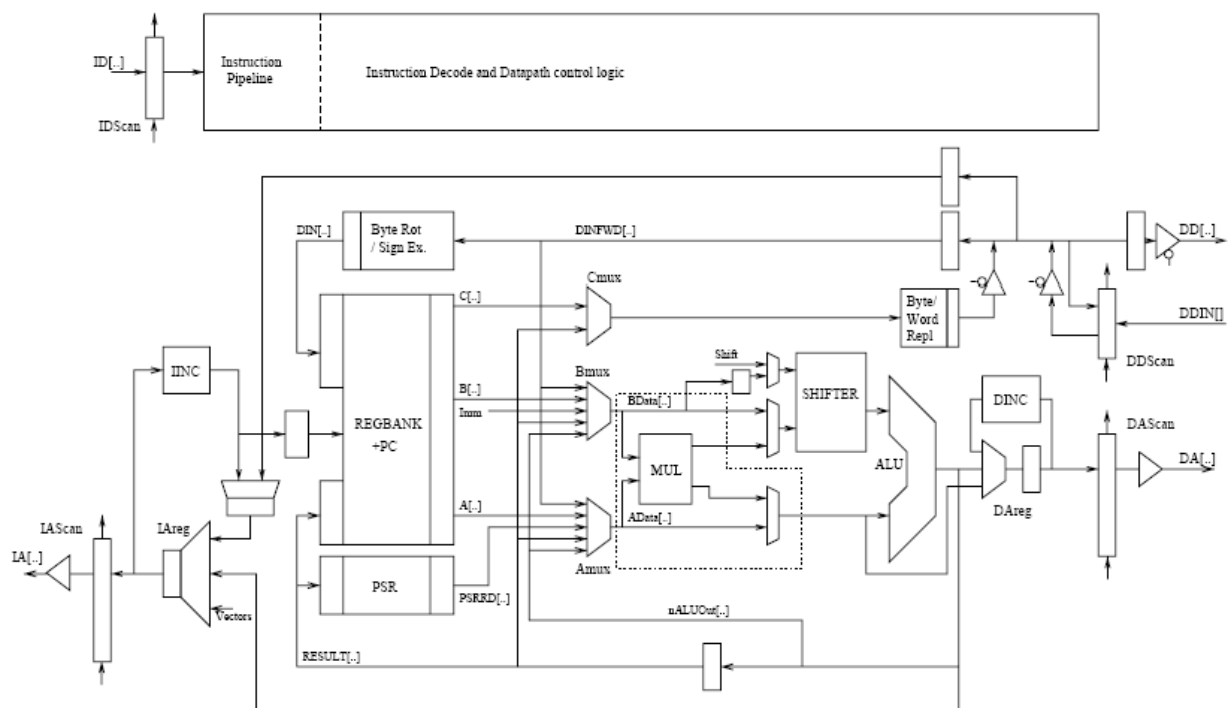


Figure [4] ARM ARCHITECTURE BLOCK DIAGRAM

Below are the technical specifications of Armv9 Architecture:

2.1. Type: RISC (Reduced Instruction Set Computing)

Armv9 is a type of RISC because it is built on the Reduced Instruction Set Computing (RISC) architecture, which uses a simplified set of commands to allow for greater energy efficiency and power savings. While modern versions like Armv9 have added advanced features for performance, they maintain the core

RISC principle of having a smaller instruction set compared to CISC architectures, which require more complex and power-intensive decode logic. This fundamental design choice is what allows Arm processors to be efficient, a key advantage for mobile devices and increasingly for high-performance computing and AI workloads.

2.2. Instruction Set Features

The Armv9 Instruction Set Architecture (ISA) builds upon the foundation of AArch64 (64-bit) introduced in ARMv8, and brings new capabilities focused on performance, scalability, machine learning, vector processing, and security.

Armv9's instruction set emphasizes simplicity, parallelism, and extensibility, consistent with the RISC design philosophy.

2.2.1. AArch64: The Foundation of Armv9

Armv9 fully adopts the AArch64 execution state, which was introduced in ARMv8.

This 64-bit mode supports:

- 64-bit general-purpose registers (GPRs) for arithmetic and addressing.
- Large physical and virtual address spaces (up to 52-bit physical addressing).
- Efficient handling of large datasets and complex workloads (e.g., cloud, AI).

Armv9 drops AArch32 support for most designs, simplifying instruction decoding and increasing efficiency.

Key Characteristics of AArch64:

- 64-bit data paths and registers.
- Fixed-length 32-bit instructions (for consistent decoding).
- Load/store architecture — memory is only accessed by dedicated instructions.
- Support for modern virtualization, exception levels, and system registers.

2.2.2. Load/Store Architecture

Armv9, like its predecessors, adheres to the load/store principle, where:

- All data-processing operations are performed on registers.
- Memory is accessed only through dedicated load (LDR) and store (STR) instructions.

This model simplifies pipeline design and improves instruction throughput.

Benefits of Load/Store Architecture:

- Reduced instruction complexity → faster decode/execution.
- Easier pipelining → fewer data hazards.
- Predictable timing and easier compiler optimization.
- Improved energy efficiency due to smaller control logic

EXAMPLE:

LDR X0, [X1] // Load value from memory into register X0

ADD X0, X0, #1 // Increment the register value

STR X0, [X1] // Store updated value back to memory

2.2.3. SIMD and NEON Extensions

Before the introduction of SVE, ARM provided NEON SIMD (Single Instruction, Multiple Data) extensions.

NEON allows a single instruction to operate on multiple data elements simultaneously — ideal for multimedia, signal processing, and vector arithmetic.

Key Features of NEON:

- 128-bit SIMD registers (V0–V31).
- Parallel arithmetic on 8-, 16-, 32-, or 64-bit data types.
- Commonly used in graphics, image processing, and audio processing.

However, NEON had fixed vector lengths (128-bit) — limiting scalability in AI and HPC contexts, which led to the evolution of SVE and SVE2.

2.2.4. SVE (Scalable Vector Extension)

Introduced in ARMv8.2-A, the Scalable Vector Extension (SVE) brought a new paradigm for vector processing. Unlike traditional SIMD, SVE uses variable vector lengths — from 128 to 2048 bits, in increments of 128 bits.

This design makes compiled code vector-length agnostic (VLA) — it automatically scales to any hardware vector width.

Key Features of SVE:

- Vector-Length Agnostic (VLA) execution: same code runs on CPUs with different vector lengths.
- Predication support: allows conditional vector operations without branch divergence.
- Gather/Scatter memory access: directly handles non-contiguous data in memory.
- Designed for scientific computing, machine learning, and data-parallel workloads.

Example	Use	Case:
In AI workloads, SVE can perform operations on matrices and tensors in parallel — increasing throughput without requiring code changes for each processor.		

2.2.5. SVE2 (Scalable Vector Extension 2)

Armv9 introduces SVE2, a major enhancement over SVE, combining the vector scalability of SVE with the versatility of NEON.

Key Features of SVE2:

- Backward compatibility with NEON while maintaining scalability.
- Extended support for integer, fixed-point, and bitwise operations.
- Enhanced vectorization for signal processing, cryptography, AI, and ML workloads.
- Supports predicated execution for conditional operations.
- Adds horizontal reductions, complex-number arithmetic, and polynomial operations.

Applications of SVE2:

- AI/ML Inference: Vectorized matrix operations for tensor processing.
- 5G Signal Processing: Efficient handling of massive MIMO and FFT operations.
- Cryptography: Parallel AES, SHA, and polynomial multiplication.

Example:

EORV B0, P0, Z1.B, Z2.B // XOR two vectors with predication mask

ADDV D0, P0, Z3.D // Vector addition with reduction

2.2.6. SME (Scalable Matrix Extension)

Introduced in Armv9.2, the Scalable Matrix Extension (SME) builds upon SVE2 to accelerate matrix-based computations, especially in AI, ML, and scientific workloads.

Core Features:

- Adds Matrix Multiply (MMLA) instructions for efficient GEMM (General Matrix Multiply) operations.
- Allows direct accumulation of matrix results without redundant memory access.
- Supports streaming mode to handle large continuous datasets efficiently.
- Reduces bandwidth pressure and improves memory locality.

Benefits:

- Greatly improved performance for neural network training/inference.
- Reduced energy consumption for high matrix-load operations.
- Foundation for ARM-based AI accelerators and HPC systems.

2.2.7. Security-Related Instruction Extensions

Armv9 integrates security mechanisms directly into the instruction set to protect against common attack vectors.

Major Security Instruction Extensions:

- **PAC (Pointer Authentication Code) Instructions:**
 - PACIA, PACIB, AUTIA, AUTIB – used to generate and authenticate pointer signatures.
- **BTI (Branch Target Identification) Instructions:**
 - Marks valid indirect branch targets, ensuring control flow integrity.
- **Memory Tagging Extension (MTE):**
 - Adds tags to memory regions and pointers to detect out-of-bounds and use-after-free errors.

Example:

PACIA X0, X1 // Authenticate pointer in X0 using key A

AUTIA X0, X1 // Authenticate pointer before use

BTI J // Define a valid branch landing pad

2.2.8. Floating-Point and Advanced Arithmetic Instructions

Armv9 continues to support IEEE-754 compliant floating-point arithmetic for single, double, and half precision.

Enhancements include:

- **Fused Multiply-Add (FMA):** Improves accuracy and speed in floating-point calculations.
- **Complex-number arithmetic:** Useful for FFTs and digital communications.
- **Polynomial instructions:** Aid in encryption and finite field arithmetic.

These enable high precision in scientific, engineering, and multimedia computations. [4]

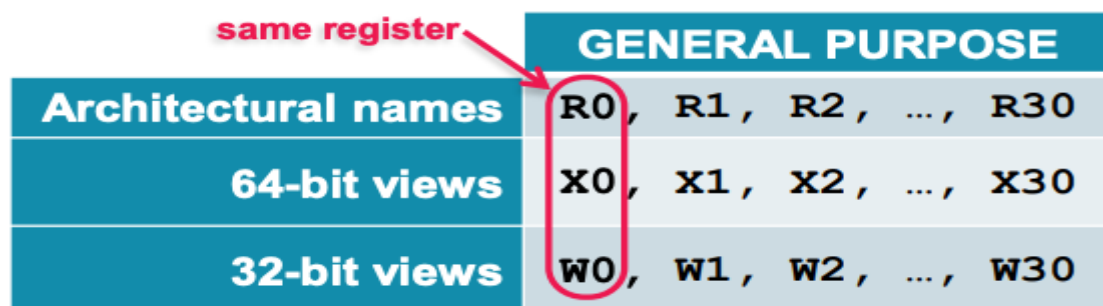
2.3. Register File (Armv9)

The AArch64 ISA defines 31 *general purpose registers* which can be used to store 64 or 32-bit values and you may find them referenced either

as X_n , W_n or R_n (capitalization is optional), where n is the index of register (a number from 0 to 30).

- When used to store 64 bits they are referenced with the letter X
- When used for 32 bits they are referenced with the letter W
- or more generally (irrelevant to the size) you may find them referenced with the letter R

For example, to refer to the *lower part of the register with 0 index* use the symbol $W0$ and for all the 64 bits you may use $X0$:



	GENERAL PURPOSE
Architectural names	R0, R1, R2, ..., R30
64-bit views	X0, X1, X2, ..., X30
32-bit views	W0, W1, W2, ..., W30

Figure [5] Armv9 InstructionSetOverview

So, this conventional rather than constructional related usage, defines the following rules:

- **Registers R0 to R7:** are used to save arguments when calling a function (see below), while the R0 is used also to store the result which is returned by a function.

Note: When a function is called, the compiler uses, a stack frame (allocated within the program's runtime stack) in order store all the temporary information that the function requires to operate. Depending on the *calling convention* the caller function will place this information to specific registers or in the *stack* or in *both*. For example the C calling convention (cdecl in short), places up to six arguments to the RDI, RSI, RDX, RCX, R8 and R9 registers and anything additional is placed into the stack.

- **Register R8:** (*Indirect result location register*), used in C++ for returning non-trivial objects (set by the caller).
- **Registers R9 to R15:** (known as *scratch registers*) can be used any time without any assumptions about their contents.

- **Registers R16, R17:** (*intra-procedure-call temporary registers*) the linker may use these in PLT code. Can be used as temporary registers between calls.
- **Register R18:** (*platform register*) reserved for the use of platform ABI. For example, for the windows ABI(Application Binary Interface), in kernel mode, points to KPCR for the current processor; in user mode, points to TEB.
- **Registers R19-R28:** can also be used as scratch registers, but their contents must be saved before usage and restored afterwards.
- **Register R29:** is used as a *Frame Pointer*, pointing to the current stack frame and it is usually useful when the program runs under a debugger. The GNU C compiler has an option to use R29 as a general purpose register via the *-fomit-frame-pointer* option.
- **Register R30:** is known as the *link register (lr)* and it can be used to store the return address during a function call, an alternative of saving the address to the call stack. Certain branch and link instructions store the current address to the link register before the program counter (the register that holds the address of the next instruction) loads the new address.
- The **stack pointer (sp)** stores the address corresponding to the end of the stack (or as commonly said pointing to the top of the stack). This address will change when registers are pushed into the stack or during the allocation or deletion of local variables.
- The **program counter (pc)** holds the next address which contains the code to be executed. It is increased automatically by four after each instruction while it can be modified by a small number of instructions (e.g. *adr* or *ldr*). This gives the ability of jumping to a new address and start executing code from there.
- The **zero register** (referenced as zr, xzr or wzr) is a special register that is hard-wired to the integer value 0. Writing to that register is always discarded and reading its value will always result in a 0 being read:

- when used as source register it always returns the integer value zero.

```
MOV    W0, #0           ; W0 = 0
MOV    W0, WZR          ; W0 = 0, same effect as previous instruction
```

- when used as a destination register it discards the value

```
SUBS   WZR, W10, W11    ; Does W10 - W11, set the flags and discard the result
CMP     W10, W11         ; Compare two numbers: CMP is an alias for the SUBS above
```

 Two ways of writing the same instruction

Figure [5] Armv9 InstructionSetOverview

- The PSTATE register is a collection of fields which are mostly used by the OS. The user programs make use of the first four bits, which are marked as N,Z,C and V respectively (referred as *condition flags*) where each one of them is interpreted as follows:

N → Negative , the flag is set to 1 when the (signed) result of an operation is negative.

Z → Zero, the flag is set to 1 when the result of an operation is 0 and set to 0 when the result is not 0.

C → Carry, the flag is set to 1 if an add operation results in a carry or a subtraction operation results in a borrow.

O → Overflow, set to 1 if a signed overflow occurs during an addition or subtraction.

- **Conditional flags:** N, Z, C, V
- **Interrupt masks:** D, A, I, F
- **Execution state:** nRW (32/64-bit)
- **Stack selection:** SP (SP_EL0/SP_ELn)
- **Misc:** SS (single step), IL (illegal execution)

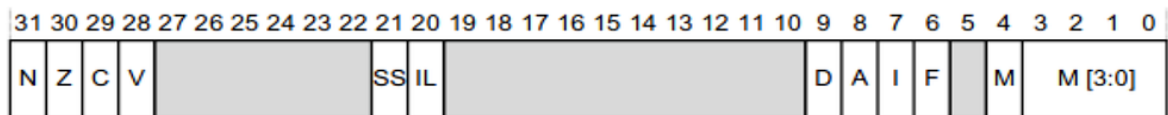


Figure [5] The PSTATE register

2.3.1 The AArch64 Registers of an Apple Silicon Macs (M1):

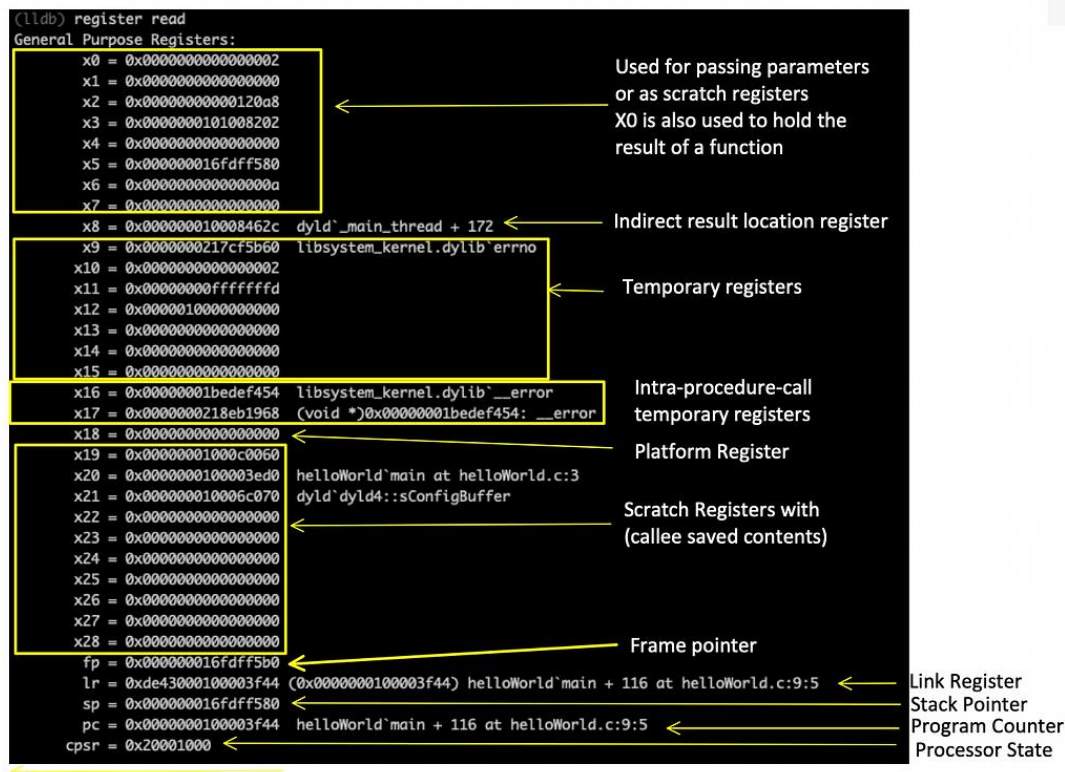


Figure [5] AArch64 Registers of an Apple Silicon Macs

The AArch64 architecture also supports 32 floating-point/SIMD registers, numbered from 0 to 31, where each can be accessed as a full 128 bit value (using v0 to v31 or q0 to q31), 64-bit value (using d0 to d31), 32-bit value (using s0 to s31) as a 16-bit value (using h0 to h31), or as an 8-bit value (using b0 to b31). Referencing less than 128 bits only access the lower bits of the full 128-bit register while leaving the remaining bits untouched unless specified otherwise. [5]

2.4. Memory Hierarchy

The memory hierarchy in Armv9 is designed to achieve high performance, low latency, and efficient power use, while also maintaining strong isolation and security guarantees. It includes multiple levels of caches, Translation Lookaside Buffers (TLBs), virtual memory, and security-aware memory management features.

Armv9 builds upon ARMv8's AArch64 memory model, extending it with improved consistency models, Realm Memory Extensions, and enhanced cache coherency for modern multi-core and heterogeneous systems.

2.4.1. CPU Registers

The fastest memory, located directly within the CPU. They store data that is immediately being processed.

2.4.2. Cache Memory

A small, fast memory that acts as a buffer between the CPU and main memory. It stores frequently accessed data from main memory.

- **Level 1 (L1):** Smallest and fastest cache, often split into instruction and data caches. It has the lowest latency.
- **Level 2 (L2):** Larger than L1, but slower. It can be private to each core or shared.
- **Level 3 (L3):** The largest and slowest on-chip cache, typically shared by all CPU cores.

2.4.3. Main Memory (RAM)

Primary, high-capacity memory where the operating system and running applications are loaded. It is slower than cache but much faster than secondary storage.

2.4.4. Secondary Storage

The slowest and largest tier, including devices like SSDs and HDDs. This is used for long-term data storage that is not in immediate use.

Key principles

2.4.5. Speed vs. Capacity

Each level trades off speed for capacity. The closer the memory is to the CPU, the faster and smaller it is.

2.4.6. Data Location

The hierarchy works by keeping the most frequently used data in the fastest levels (registers and caches) to reduce latency and improve performance.

2.4.7. Cache Management

When the CPU needs data, it first checks the L1 cache. If it's not there (a "miss"), it checks the next level (L2), and so on. When data is found, it is copied into the faster levels to speed up future access. [6]

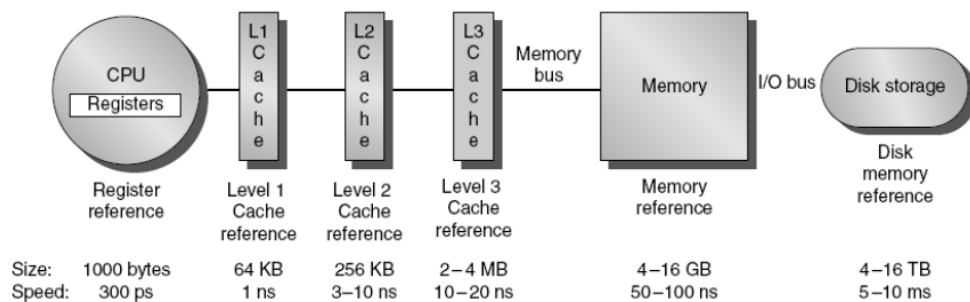


Figure [6] Memory Hierarchy

2.5. Pipelining and Parallel Execution

Pipelining in Armv9 builds upon the five-stage pipeline found in older cores like the ARM9, but Armv9 also employs advanced parallel execution techniques such as the Scalable Vector Extension (SVE2) and hardware transactional memory (TME) to further boost performance and efficiency. The five-stage pipeline (Fetch, Decode, Execute, Memory, Writeback) allows for concurrent instruction processing by breaking it into stages. Pipelining increases throughput by overlapping instruction execution, while parallelism, like with SVE2, allows the processor to perform multiple operations simultaneously on different data, creating a true parallel execution model in addition to the pipelined one.

2.5.1. Five-Stage Pipelining (found in ARM9 and a foundation for later cores)

- **Fetch:** Instructions are retrieved from memory.
- **Decode:** The fetched instruction is decoded to identify the operation and the required operands.
- **Execute:** The Arithmetic Logic Unit (ALU) performs the operation, such as an arithmetic or logical calculation. For load/store instructions, the memory address is calculated in this stage.
- **Memory Access:** Data is read from or written to the data memory. For instructions not accessing memory, the result is buffered.

- Write Back: The final result is written back to the appropriate register.



Figure [8] Single Execution Cycle

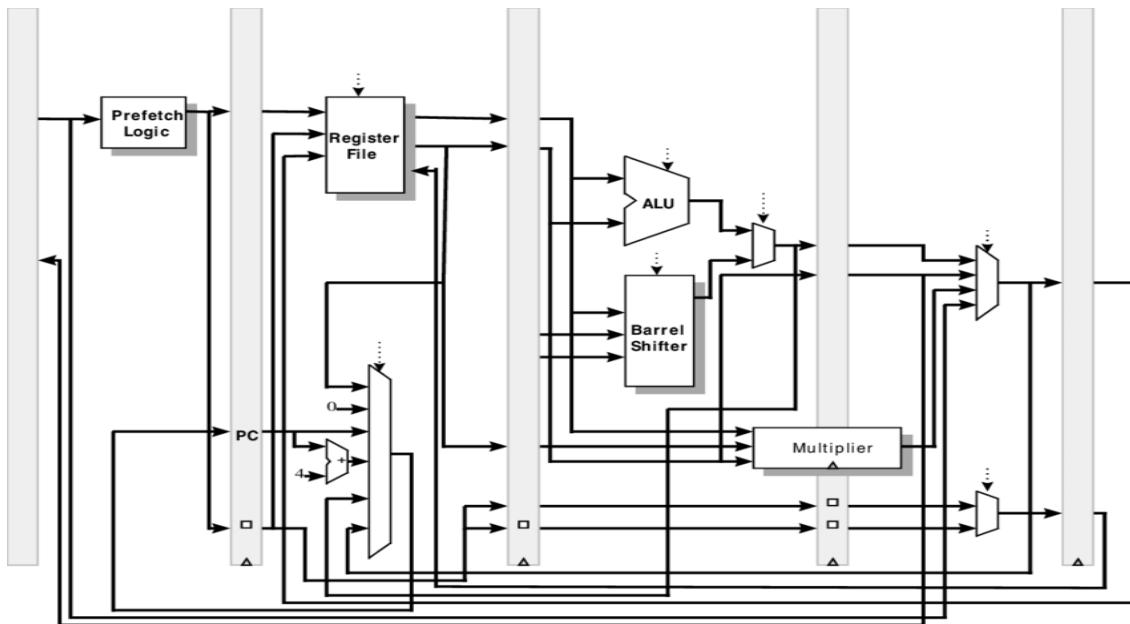


Figure [8] Pipelining in Armv9

2.5.2. Parallel execution in Armv9

- Scalable Vector Extension (SVE2):

This extends the ability to perform parallel operations to a wider range of use cases, such as processing large datasets for genomics and computer vision.

- Hardware Transactional Memory (TME):

This is a hardware-based technique that improves the performance of multi-threaded applications by allowing multiple threads to execute in parallel within a "transaction." The hardware ensures that either the entire transaction completes successfully or none of it affects the program state, avoiding race conditions.

- Parallel Decode Units:

Some cores, like the ARM9TDMI, implemented parallel decode units for ARM and Thumb instructions, allowing them to decode different instruction sets in parallel.

- Pipelining vs. Parallelism:

Pipelining is a form of parallelism where different stages of a single instruction are executed simultaneously. Parallelism can also refer to executing multiple independent instructions at the same time, which is what technologies like SVE2 and TME enable in addition to the core pipeline.

2.6. Power and Performance Optimizations

Power and performance optimizations in Armv9 leverage new CPU microarchitecture designs, enhanced memory systems, and specialized instructions like SVE2 to improve performance, efficiency, and security. Key optimizations include the use of heterogeneous computing in big.LITTLE architectures (e.g., Cortex-X and Cortex-A series cores), targeted updates for AI and gaming workloads, and increased memory bandwidth and reduced latency. Security improvements, such as the Confidential Compute Architecture (CCA) and memory tag extensions, also contribute to performance by ensuring greater integrity and reliability.

2.6.1. Performance optimizations

- CPU microarchitecture:

Armv9 includes new CPU cores with significant performance uplifts compared to the previous generation, such as the Cortex-X925, which offers a 36% single-threaded performance boost.

- Memory and cache:

The architecture aims to reduce memory latency and improve CPU frequency bandwidth and cache performance to handle large datasets more efficiently.

- Heterogeneous computing:

The big.LITTLE architecture uses high-performance "big" cores (like Cortex-X and Cortex-A720) for demanding tasks and highly efficient "LITTLE" cores (like Cortex-A520) for background tasks, balancing power and performance.

- Vector processing:

The baseline inclusion of Scalable Vector Extension 2 (SVE2) provides more efficient and flexible parallel processing without requiring applications to be recompiled for different vector sizes.

- Infrastructure:

Infrastructure-focused CPUs, such as the Neoverse N2, deliver up to a 40% IPC performance boost over the previous generation while maintaining high power efficiency.

2.6.2. Power optimizations

- Efficiency-focused cores:

The Cortex-A520 is designed for maximum energy efficiency, providing 15% better efficiency than its predecessor.

- Targeted optimizations:

Microarchitecture optimizations are specifically focused on improving power efficiency, with the Cortex-A715 achieving 20% power efficiency improvements over its predecessor while maintaining performance.

- DynamiQ technology:

This technology allows for flexible cluster configurations of big.LITTLE cores to be optimized for specific power and performance targets. [8]

2.7. Security Features (Confidential Compute Architecture)

Armv9 includes the Confidential Compute Architecture (CCA), which introduces Realms to protect data in use by creating isolated environments. Key security features include the Realm Management Extension (RME), which enables realms to have their own memory, and an added Root world to manage the new execution states (normal, secure, and realm). This allows sensitive data to be processed securely even from privileged software like the hypervisor or operating system.

2.7.1. Confidential Compute Architecture (CCA)

- Realms:

Isolated execution environments for confidential computing, which shield applications and their data from higher-privilege software.

- Hardware-backed security:

CCA leverages hardware extensions to provide strong isolation and encryption for realms, reducing the need to trust the hypervisor or other parts of the system.

- Attestation:

Enables realms to be attested, providing a way for a relying party to verify the trustworthiness of the environment.

2.7.2. Core components

- Realm Management Extension (RME):

The hardware foundation for CCA, providing the necessary components for isolation, attestation, and dynamic resource management.

- Realm Management Monitor (RMM):

A firmware component that enforces security constraints and manages resource policies for realms.

- New worlds:

Introduces new security states—Realm and Root—alongside the existing Normal and Secure worlds, managed by the Root world's monitor. [1]

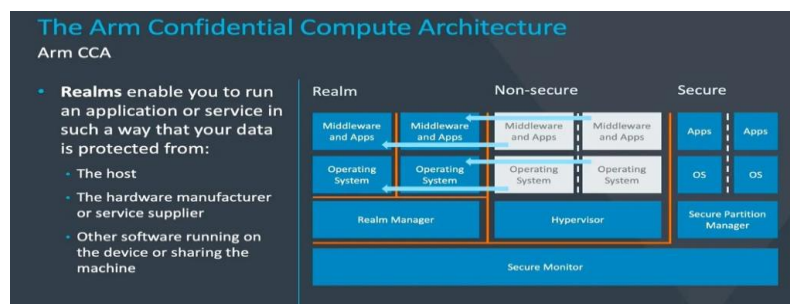


Figure [1] Armv9 CCA

3. Armv9 Comparisons: Overview and Key Features

3.1. Comparison with Previous Architectures

Armv9 greatly improves the vector processing capabilities of earlier versions by introducing Scalable Vector Extension 2. This enhances parallelism,

adds support for fixed-point arithmetic, and is very effective in AI, DSP, and ML. Armv9 also boosts performance in areas such as genomics and computer vision with the Scalable Matrix Extension. SVE2 offers variable vector lengths of up to 2048 bits for different data types. This allows for high throughput in operations on large data while maintaining power efficiency.

In comparison, ARMv8's vector support was much weaker, depending on SVE in some variants and NEON SIMD units. These only allowed for modest vector throughput when supported. Armv9's SVE2 and SME represent a significant advancement, enabling comprehensive AI processing and improved parallelism [12].

Intel and AMD processors based on the x86 architecture support demanding workloads in AI and ML through AVX, AVX2, AVX-512, and newer AMX instructions. These instruction sets deliver strong floating-point performance and parallel computation, often at the expense of higher power consumption. Recent designs, like Intel's Meteor Lake, have added Neural Processing Units to speed up various AI applications for both consumer and edge environments [13].

3.2. Feature Comparison

3.2.1. AI / ML and Vector Extensions

Armv9 advances vector processing by introducing Scalable Vector Extension 2, which increases parallelism and supports fixed-point arithmetic. This has proven to be very effective in AI, DSP, and ML. Additionally, the Scalable Matrix Extension significantly enhances performance in genomics and computer vision. SVE2 supports vector lengths of up to 2048 bits for different data types, ensuring high throughput and power efficiency for operations on large datasets [12].

In contrast, ARMv8 had more limited vector capabilities, relying on SVE in some variants and NEON SIMD units. These offered medium-range vector performance when available. The switch to Armv9's SVE2 and SME marks a leap forward, allowing for full-scale AI processing and better parallel efficiency [12].

Intel and AMD's x86 architecture supports demanding workloads in AI and ML through AVX, AVX2, AVX-512, and newer AMX instructions. These extensions deliver high floating-point and parallel computation, primarily at the cost of increased power usage. Recently, designs like Intel's Meteor Lake have included

Neural Processing Units to enhance various AI tasks for consumer and edge applications [13].

3.2.2. Performance Comparison of x86, ARM, and RISC-Processors

As AI/ML continues to evolve, choosing the right processor architecture is more important than ever. It’s essential to balance performance, efficiency, and flexibility. The three main contenders, x86, ARM (ARMv8 and Armv9), target different AI applications [13].

Intel and AMD x86 processors provide substantial computing power and strong floating-point performance. Their instruction sets promote parallel efficiency through AVX, AVX2, and AVX-512. Newer chips, such as those from Intel Meteor Lake, also include integrated NPUs that accelerate AI models. However, their high power consumption makes them unsuitable for mobile or embedded systems.

The ARM architectures focus on efficiency without sacrificing performance. ARMv8 introduced 64-bit computing, TrustZone security, and virtualization, while Armv9 adds SVE2, Scalable Matrix Extension, and Confidential Compute Architecture for advanced AI and ML workloads. These features make Armv9 particularly well-suited for edge and mobile AI applications. In summary, x86 is best for high-performance AI applications, while ARMv8 and Armv9 excel in efficient AI for edge devices.

Category	x86	ARM
GPU	Typically equipped with powerful integrated GPUs (e.g., Intel or AMD Radeon), delivering strong graphics and AI/ML performance	Often use ARM Mali GPUs or third-party solutions; suitable for mobile and embedded AI tasks but with lower overall GPU power.

NPU	Often rely on external accelerators (e.g., NVIDIA Tensor Core) for high AI compute power; some, like Intel Meteor Lake, integrate NPUs for on-device AI.	Combine with ARM Ethos NPUs for efficient, low-power AI acceleration; well-suited for edge and embedded AI.
Image Processing	Strong image processing performance due to advanced integrated graphics; ideal for complex visual AI tasks.	Paired with powerful Image Signal Processors (ISPs), delivering efficient image processing for mobile AI.
Supported Network Architectures	Broad support for CNNs, RNNs, Transformers, GANs, and other AI networks; mature frameworks and libraries.	Strong compatibility with AI frameworks like TensorFlow, PyTorch, and Caffe; optimized for embedded applications.

3.2.3. Security Features

The Armv9 architecture introduces a comprehensive hardware-based security model for modern computing. Its Realm Management Extension (RME) enables Confidential Computing through isolated “realms” that secure data from privileged access. Pointer Authentication (PAC) and Branch Target Indicators (BTI) protect control flow, while Memory Tagging Extension (MTE) prevents common memory errors [12].

ARMv8 established the foundation with TrustZone for secure execution, Secure Boot for firmware integrity, and protections like DEP and ASLR. It also integrated

cryptographic engines, true random number generation (TRG), and software isolation to enhance reliability.

x86 systems (Intel and AMD) implement layered security through technologies like Intel SGX, AMD SEV, CET, TPM, and Secure Boot. However, these are additive features built over time rather than a unified design.

In summary, Armv9 offers the most integrated security model, ARMv8 provides a strong base and x86 remains secure but relies on retrofitted mechanisms [12].

3.2.4. Instruction Set / ISA Philosophy

The x86 architecture follows the Complex Instruction Set Computing (CISC) model, using variable-length instructions that perform multiple operations per command. This simplifies programming but increases hardware complexity and power usage. Despite this, x86 remains dominant in high-performance and general-purpose computing due to its power and compatibility [14].

ARMv8 and Armv9 use the Reduced Instruction Set Computing (RISC) approach with fixed-length instructions for efficient decoding and lower energy use. ARMv8 introduced 64-bit AArch64 processing, while Armv9 added AI acceleration and advanced security, making ARM ideal for mobile and low-power systems [14].

Overall, x86 prioritizes power and compatibility, while ARM focus on efficiency and adaptability [14].

3.2.5. Cost and Availability

ARM processors have a clear advantage in cost and availability, widely used in mobile, embedded, and IoT devices. Large-scale production and low manufacturing costs make them ideal for diverse applications [13].

x86 processors are generally more expensive and less available in some markets. They dominate high-performance computing, data centers, and PCs, where higher costs match their performance demands [13].

3.3. Real Life Comparison

3.3.1. x86

Intel Core and AMD Ryzen processors dominate desktops, laptops, and data-center servers, offering high performance and strong backward compatibility.

3.3.2. ARM

Apple’s M-series chips (M1–M4) and Qualcomm Snapdragon SoCs use ARMv8 and Armv9 architectures to deliver exceptional energy efficiency for mobile and laptop devices.

3.3.3. RISC-V

SiFive Performance P870 and Alibaba Xuantie C910 represent emerging high-performance RISC-V chips, highlighting the growing adoption of open-source hardware design.

3.4. Summary Table

3.4.1. ARMv8 vs Armv9

Feature	ARMv8	Armv9
Architecture	64-bit (with backward 32-bit compatibility)	64-bit only (focus on enhanced performance and security)
Performance	High efficiency and power optimization	Improved IPC, higher clock scalability, and better parallel processing

AI / ML Support	NEON SIMD and optional SVE (vector processing)	Advanced SVE2 (Scalable Vector Extension v2) for ML, DSP, and 5G workloads
Security	TrustZone and Secure Boot	Confidential Compute Architecture (CCA) for hardware-level isolation and better data protection
Virtualization	Full support introduced	Enhanced virtualization with finer-grained memory protection

3.4.2. x86 vs ARM

	x86	ARM
Origin	Intel and AMD	Arm Ltd.
Instruction Set	CISC (Complex Instruction Set Computing)	RISC (Reduced Instruction Set Computing)
Byte Order	Little-endian	Typically bi-endian (user-configurable)
Applications	Desktops, laptops, servers, workstations	Mobile devices, embedded systems, servers
Licensing Model	Intel and AMD produce their chips	ARM licenses its designs to manufacturers
Ecosystem	Large software and hardware ecosystem	Large ecosystem, extensive third-party support

4.

Application Domains

Armv9 the new generation of computing architecture. The main focus of this powerful device is on efficiency, and computing is found in almost all devices from our hand held smartphones to the giant computers in the data centres [17].

4.1. Mobile Computing (Smartphones)

Today all our latest smartphones are powered by Armv9. Armv9 is better choice as they deliver efficiency and better speed . This instruction set architecture

powers all the work seamlessly from high powered camera sensors to extended battery life to typical medium to high powered applications[17]. There is also an advancement of Confidential Computer Architecture (CCA) which create isolated zones in the processor which help in protecting sensitive informations like biometrics and fingerprints keeping them protected from main operating system and other apps even if the device is compromised.

Real Life Examples: Armv9 is found in Qualcomm's Snapdragon 8 Gen series (in most of the high-end Android phones), Samsung Exynos chips, and the incredibly powerful Apple A-series chips in its iPhones and Apple M-series chips in its iPads.

4.2. Laptop and personal computing

Armv9 is putting now pressure on traditional laptop processors with a favorable combination of chips and battery life. Laptops based on this architecture can be very thin and light, often with no fan, wasted energy, and still deliver lots of power for work, school, leisure, works and even video editing [17].

Real world examples include the Apple MacBook Air and MacBook Pro with M-series chips which are known and praised for their silence and day-long battery. Now new Windows laptops with the Qualcomm Snapdragon X Elite processor introduce similar benefits to the PC industry.

4.3. Data Centers and Cloud Computing

In the field of the huge server that comprise "the cloud," Armv9 is entering. Its terrific performance per watt allows companies to build server farms that consume less power, while still being able to outperform existing server farms. A primary technology in this context is SVE2 (Scalable Vector Extension 2), which enables the CPU to process large data vectors in parallel, ideal for AI data modelling or big data workload analysis [15]. CCA here allows the cloud servers to run the customer software in such confidential way that its information is encrypted and invisible to the cloud server's own host operating system.

Examples in the Real World: Amazon Web Services (AWS) is running many of its cloud service stacks on their own Graviton3 and Graviton4 processors that are based on Arm. Other players including Azure and Google Cloud, are also offering service stacks based on Arm chips, including chips from companies like Ampere Computing.

4.4. AI and Machine Learning

Armv9 has capabilities that make it very well-suited for AI workloads. The technology of SVE2 is the main factor here. This allows the chip to process many data points simultaneously, which is advantageous for doing advanced mathematics often found in machine learning [15]. This, in turn, starts difficult forms of AI work like image recognition or NLP to run on device, which is faster and more power consuming.

Real Life Examples: Many of the AI powered features we see on our mobile phones today such as translation of other languages , taking high quality portrait mode photos and having an agentic voice assistant are all using Armv9 and sve2. The technology is being utilized in AI accelerators and edge computing devices created by business partners such as Nvidia for solutions such as robotics and smart city cameras.

4.5. Automotive and Advanced Driver-Assistance Systems (ADAS)

Today's vehicles are really computers on wheels Armv9 provides the compute performance required. It powers the interactive displays for navigation and entertainment features and most importantly, processes the information from cameras and sensors to facilitate safety features such as automatic emergency braking and advanced cruise control. SVE2 enables the chip to quickly analyze information from multiple sensors (cameras, radar, and etc.) simultaneously, which is critically important for safely driving a vehicle [15]. CCA here is also used to create hardware related “firewall” for safety critical systems.

Real-World Examples: The leading automotive chip suppliers including Nvidia, Renesas and Texas Instruments all use Arm technology in their platforms - these are the advanced driver assist capabilities in the cars of manufacturers such as Tesla, Ford, and Toyota.

4.7. Conclusion

Arm V9’s advantage lies in its scalable architecture and SVE2 support, which sets Arm up in unique position across mobile, cloud, AI and automotive use case workloads. It starts with power efficiency and dedicated compute capabilities from advanced compute capabilities to data centres, while processing security and performance.

5. Future Prospects

The initiation of Armv9 is not merely an enhancement but a foundation for the next decade of computing. Armv9 is built on performance, security, and dedicated computing. Armv9 is expected to cause changes across the technological spectrum. Armv9 will evolve in three ways: seamless integration of AI everywhere, agreement around a security standard, and an ever-expanding assault on the world of traditional computing.

5.1. Pervasive and Specialized AI from Cloud to Edge

The evolution of computing is inextricably tied to processing AI workloads efficiently. Armv9's SVE2 and forthcoming improvements to matrix multiplication performance are only the start. We will begin to see these technologies migrate from high-performance servers into every tier of the device ecosystem. As Simon Segars, the former CEO of Arm, said, the roadmap “supports specialized processing” to suit the demands of various AI tasks [17]. This means:

5.1.1. Smarter Edge Devices

When released, the Armv9-M profiles will provide sophisticated AI capabilities to the most power-constrained endpoint devices. For example, intelligent sensors in agriculture, healthcare, and smart industrial IoT will be able to locally run complex ML models, like the tuberculosis detection example run locally on Cortex-M55 [17], while having an intelligent sensor to process the data, all without accessing the cloud at potentially any point. The benefits for running these locally include reduced latency, saved bandwidth, and improved privacy.

5.1.2. Ubiquitous AI in Consumer Tech:

The future of smartphones, laptops, and XR headsets will come equipped with Armv9's AI accelerations allowing for more intelligent personal assistants, real-time language translation, and enhanced augmented reality experiences while preserving Arm's defined power efficiency.

5.2. Redefining Security with Confidential Compute as Standard

With millions of IoT attacks occurring every day, the evolving threat landscape means security is not just a feature anymore [17]. Arm's Confidential Compute Architecture (CCA) will be the basis of trust for tomorrow's digital system.

5.2.1. Industry-Wide Adoption

CCA will transition from being a premium feature to an expected default feature of processors. This will establish hardware-enabled "realms" of protection that will isolate and protect sensitive data from health information about an individual to the proprietary algorithm of a business for violation of the safety of the entire software stack including the operating system.

5.2.2. Expanding the Security Perimeter

Future CCA versions will most likely expand from CPU cores to cover GPUs, NPUs, and system components, offering an end-to-end confidential compute environment. This will play an important role in protecting data in multi-tenant cloud environments and in vehicle-to-everything (V2X) communication.

5.2.3. Eliminating Memory Safety Vulnerabilities

The Memory Tagging Extension (MTE) will significantly reduce entire classes of memory safety bugs (such as buffer overflows), which have resulted in most software exploits for decades. This leads to a more secure basis for all software, which includes operating systems and applications.

5.3. Challenging and Replacing Incumbent Architectures

Armv9 is the single biggest threat to the longstanding x86 supremacy in performance-intensive domains.

5.3.1. The Cloud and Datacenter

The performance per watt advantage of Arm-based processors, such as AWS Graviton and Arm Neoverse series, enhanced with Armv9 SVE2 and AI capability, will continue to be enticing for cloud providers and enterprises. The fact that Fugaku, the number 1 supercomputer on the most recent Top500 list, has proven the Arm-based SVE capabilities for the most demanding workloads for both HPC and AI. In the future, we expect Armv9 to capture a much larger share

of the server market, especially in scale-out datacenters as well as with workloads like AI inference and data analytics.

5.3.2. The Client PC Market

With the success of Apple's M-series chips, we've seen Arm is capable of superior performance and battery life in laptops. With Qualcomm's Snapdragon X Elite and future Armv9-based platforms, we can expect to see a wave of Windows laptops, some with what will be described as "all day" battery life, always-on connected capabilities, and better AI capabilities, directly challenging the x86 dominance in the personal computing space.

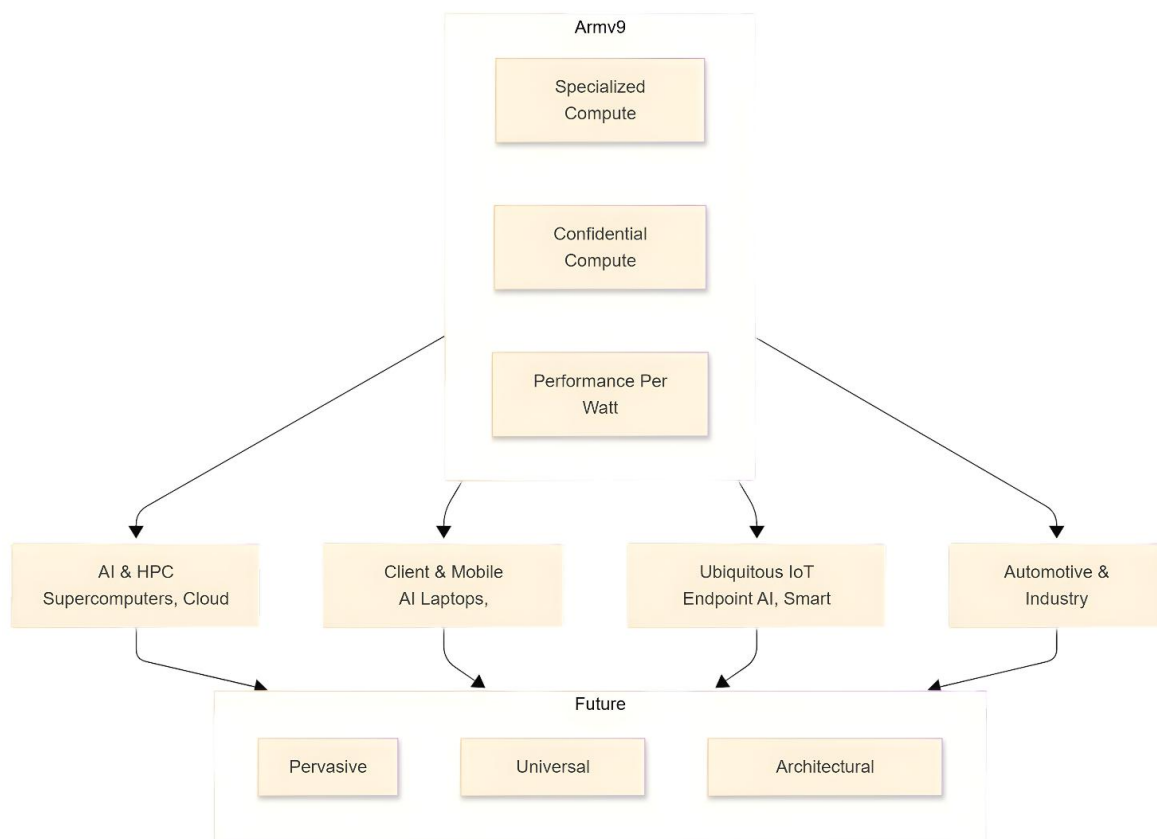


Figure Armv9 Foundation

5.3.3. The Total Compute Design Approach for Arm

Armv9 adopts a Total Compute approach to enhance performance not just within the CPU, but across the entire system, from CPUs to GPUs and NPUs, and system interconnects. Arm expects to see total system performance outweighed by CPU performance improvements of over 30% in the next two mobile or infrastructure CPU generations [18], not just from CPU performance improvements, but in suitability for real workloads for special use cases.

To help visualize the comprehensive scope of Armv9's future, the following diagram illustrates its core pillars and their applications across the computing spectrum:



Figure[18] : The Armv9 Ecosystem and Future Evolution

The Armv9 Foundation features important components: heterogeneous compute (SVE2 and Matrix Math), confidential computing (CCA and MTE), and performance efficiency. These capabilities unleash innovations in areas such as AI and HPC, client and mobile devices, IoT devices, and automotive or industrial environments. Altogether, these domains will accelerate Arm’s next wave of innovation toward pervasive AI, security for all, and the end of architectures.

5.4 Conclusion

In summary, we can conclude that Armv9 architecture is uniquely positioned to be the platform of the future. Its roadmap leads the way to meeting the increasing needs for computing that is trusted, intelligent, and efficient. Armv9 will fuel the next 300 billion chips while prioritizing specialization in AI, making confidential computers accessible to a broader audience, and naturally leveraging its performance-per-watt advantages to redefine the devices and infrastructure we use to run our world.

6. Conclusion

The Armv9 architecture holds a significant milestone in computing, purposefully designed for the age of specialized AI and ubiquitous security. With innovative features like SVE2 for scalable vector processing and the Confidential Compute Architecture for security at the hardware level, Armv9 functions at an unprecedented performance per watt level while tackling the challenges facing computing today. Its proven use in smartphones, data centers, and automotive systems powering the platform for everything from Apple's M-series to AWS Graviton demonstrates the continued superiority of the ARM architecture over prior generations and competing architectures. As the basis for the next generation of intelligent devices, Armv9 is at the center of computing evolution for the next decade, and continues to be a vital part of the technology ecosystem as the foundation for intelligent, efficient, and secure processing.

7. XOR Rotatory Password Hashing Algorithm (ARMv9 Assembly)

7.1. Introduction

The **XOR Rotatory Password Hashing Algorithm** is a lightweight cryptographic hashing technique implemented in ARMv9 assembly. It is designed to securely transform user-provided passwords into fixed-size hash values by combining bitwise rotations and XOR operations. The algorithm reads the input password byte by byte, applies a left rotation on the hash accumulator, and XORs it with each character, producing a unique hash for each input. This approach demonstrates low-level programming skills, bit manipulation, and a solid understanding of the ARMv9 instruction set.

The resulting hash is then converted into an ASCII string for display, ensuring readability while maintaining the integrity of the hashed data.

To compile and run this algorithm on a Linux system (or WSL Ubuntu) with QEMU for ARM64:

```
aarch64-linux-gnu-as -o hashing.o hashing.s
```

```
aarch64-linux-gnu-ld -o hashing hashing.o
```

```
qemu-aarch64 ./hashing
```

7.2. Technical Implementation

```
.section .data // Stores initialized data
like strings

prompt: .asciz "Enter password: " // Text shown before
taking user input

output_str: .asciz "Hashed Password: " // Label shown
before printing the hash

hexbuf: .space 32 // Buffer to hold the
ASCII digits of the final hash


.section .bss // Memory reserved but not
initialized

buffer: .skip 128 // Space for up to 128
bytes of password input


.section .text // Actual program
instructions

.global _start // Program entry point

_start:

    // Display input prompt on screen

    mov x0, 1 // Write to stdout

    ldr x1, =prompt // Address of prompt string

    mov x2, 16 // Number of bytes to write

    mov x8, 64 // Linux write syscall

    svc 0 // Execute system call
```

```

    // Read user-entered password

    mov x0, 0 // Read from stdin

    ldr x1, =buffer // Store input in buffer

    mov x2, 128 // Max bytes to read

    mov x8, 63 // Linux read syscall

    svc 0 // Execute system call

    // Start hash calculation

    mov w0, 0 // Hash accumulator starts
at zero

    ldr x1, =buffer // x1 will walk through the
input bytes

hash_loop:

    ldrb w2, [x1], #1 // Load next byte, then move
pointer forward

    cbz w2, hash_done // Stop if we reach a null
terminator

    mov w3, w0 // Temp copy for rotation

    lsl w0, w0, #5 // Rotate-left step: shift
left

    lsr w3, w3, #27 // Rotate-right step: wrap-
around bits

    orr w0, w0, w3 // Complete rotate-left
operation

    eor w0, w0, w2 // Mix current byte into
hash

    b hash_loop // Continue hashing next
byte

```

```

hash_done:

    mov w4, w0                // Keep final hash value in
w4

    ldr x5, =hexbuf          // Pointer for writing ASCII
digits

    mov w6, 0                // Reset digit counter to
zero

conv_loop:

    mov w3, 10               // We convert number to
base-10 string

    udiv w7, w4, w3          // Quotient = hash / 10

    msub w8, w7, w3, w4      // Remainder = last digit

    add w8, w8, 48           // Turn remainder into ASCII
character

    strb w8, [x5, w6, uxtw]   // Store ASCII digit

    add w6, w6, 1           // Count one more digit
added

    mov w4, w7               // Use quotient for the next
loop

    cbnz w4, conv_loop       // Continue until quotient
becomes zero

    // Reverse the string to get correct digit order

    mov w7, 0                // Start index for reversing

rev_loop:

```

```

        cmp w7, w6                                // Have we swapped
everything?

        bge print_hash                            // If yes, move on to
printing

        sub w8, w6, w7, uxtw                     // Mirror index start

        sub w8, w8, 1                             // Compute exact mirror
position

        ldrb w9, [x5, w7, uxtw]                  // Read left byte

        ldrb w10, [x5, w8, uxtw]                 // Read right mirrored byte

        strb w9, [x5, w8, uxtw]                  // Swap left → right

        strb w10, [x5, w7, uxtw]                 // Swap right → left

        add w7, w7, 1                             // Move inward

        b rev_loop                               // Repeat until done

print_hash:

        // Print label before showing final hash

        mov x0, 1                                // stdout

        ldr x1, =output_str                      // Address of label text

        mov x2, 17                               // Length of label

        mov x8, 64                               // write syscall

        svc 0                                    // Display label

        // Print the hash itself

        mov x0, 1                                // stdout

        ldr x1, =hexbuf                          // Digits buffer

        mov x2, x6                               // Number of digits

```

```

mov x8, 64                // write syscall
svc 0                     // Display hash

// Print newline for clean output
mov x0, 1                 // stdout
ldr x1, =nl               // Newline string
mov x2, 1                 // One character
mov x8, 64                // write syscall
svc 0                     // Print newline

mov x0, 0                 // Exit status
mov x8, 93                // exit syscall
svc 0                     // Terminate program

.section .data
nl: .asciz "\n"           // Single newline character

```

7.3. Code Explanation

7.3.1. Algorithm

hash = 0 ; Initialize accumulator

for each byte b in input:

hash = ROTL(hash, 5) ; Rotate hash left by 5 bits

hash = hash XOR b ; Mix the byte into hash

7.3.2. Explanation:

1. Initialize the hash accumulator to zero.
2. Process each byte of the input password sequentially.
3. Rotate the hash left by 5 bits to spread bits across the accumulator.
4. Mix in the current byte using a XOR operation, producing a unique hash for each distinct input.
5. Repeat until the entire password is processed.
6. Finally, the resulting 32-bit hash is converted to an ASCII decimal string for display.

7.4. Results

7.4.1. Simple Text

```
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: password
Hashed Password: 688942855
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: 123456
Hashed Password: 498711756
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: abc
Hashed Password: 4932123
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: hello
Hashed Password: 0503217953
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: test
Hashed Password: 681499421
```

7.4.2. Mixed Case & Numbers

```
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: xYz987
Hashed Password: 0223209403
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: HashMe2025
Hashed Password: 3972503822
```

7.4.3. Special Characters

```
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: P@ssw0rd!
Hashed Password: 437603857
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: !@#$%^&*()
Hashed Password: 1985677724
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: User#1$
Hashed Password: 030233122
```

Activate Windows

7.4.4. Longer Inputs

```
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: ThisIsAVeryLongPasswordToTestTheAlgorithm
Hashed Password: 8784579653
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: LoremIpsumDolorSitAmet1234567890
Hashed Password: 4250442791
```


7.4.5. Edge Cases

```
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password: a
Hashed Password: 4113
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password:
Hashed Password: 01
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-as -o hashing.o hashing.s
root@AlisPC:/mnt/c/Users/alira/Desktop# aarch64-linux-gnu-ld -o hashing hashing.o
root@AlisPC:/mnt/c/Users/alira/Desktop# qemu-aarch64 ./hashing
Enter password:
Hashed Password: 4301
```

8. References:

- [1] Arm Ltd., "Arm A-Profile Architecture: Armv9," 2024. [Online]. Available: <https://www.arm.com/architecture/cpu/a-profile/Armv9>. [Accessed: May 27, 2024].
- [2] T. Frick, "Armv9: The Next Generation of Processors," *IONOS Digital Guide*, May 11, 2022. [Online]. Available: <https://www.ionos.com/digitalguide/server/know-how/Armv9/>. [Accessed: May 27, 2024].
- [3] "ARM Architecture Family," *Wikipedia*, 2024. [Online]. Available: https://en.wikipedia.org/wiki/ARM_architecture_family. [Accessed: May 27, 2024].
- [4] ARM Ltd., *PrimeCell Color LCD Controller (PL110) Technical Reference Manual*, Document Number DDI0180A, 2000. [Online]. Available: <https://developer.arm.com/documentation/ddi0180/a/introduction/processor-block-diagram>. [Accessed: Oct. 26, 2024].
- [5] S. Valsamaras, "ARM 64 Assembly Series — Basic Definitions and Registers," *Medium*, Sep. 27, 2020. [Online]. Available: <https://valsamaras.medium.com/arm-64-assembly-series-basic-definitions-and-registers-ec8cc1334e40>. [Accessed: Oct. 26, 2024].
- [6] L. Null and J. Lobur, *The Essentials of Computer Organization and Architecture*, 5th ed. Jones & Bartlett Learning, 2018. [Online]. Available: https://faculty.ksu.edu.sa/sites/default/files/CAQA5e_ch2mod.pdf. [Accessed: Oct. 26, 2024].
- [7] M. A. El-Moursy, "ARM Pipeline and Main Functional Blocks," *ResearchGate*, 2004. [Online]. Available: https://www.researchgate.net/figure/ARM-pipeline-and-main-functional-blocks_fig1_3786942. [Accessed: Oct. 26, 2024].
- [8] GeeksforGeeks, "Pipelining in ARM," *GeeksforGeeks.org*, Nov. 9, 2021. [Online]. Available: <https://www.geeksforgeeks.org/digital-logic/pipelining-in-arm/>. [Accessed: Oct. 26, 2024].
- [9] IONOS Editorial Team, "What is Armv9? AI-Ready CPU Architecture," *IONOS DigitalGuide*, Apr. 9, 2025. [Online]. Available: <https://www.ionos.com/digitalguide/server/know-how/Armv9/>. [Accessed: Oct. 25, 2025].

- [10] A. Gibbons, "Armv9: The Long-Awaited High-Performance Computing Architecture," *All About Circuits News*, Apr. 1, 2021. [Online]. Available: <https://www.allaboutcircuits.com/news/Armv9-the-long-awaited-high-performance>. [Accessed: Oct. 25, 2025].
- [11] J. Smith, "ARM vs. x86: What's the Difference?" *Red Hat*, Jul. 21, 2022. [Online]. Available: <https://www.redhat.com/en/topics/linux/ARM-vs-x86>. [Accessed: Oct. 25, 2025].
- [12] ARM, "Armv9-A: New Generation of the Arm A-Profile CPU," *Arm*, 2025. [Online]. Available: <https://www.arm.com/architecture/cpu/a-profile/Armv9>. [Accessed: Oct. 25, 2025].
- [13] DFRobot, "x86, ARM, and RISC-V, How to Choose the Optimal Architecture for Your AI Applications?" *DFRobot*, Dec. 20, 2023. [Online]. Available: <https://www.dfrobot.com/blog-13494.html>. [Accessed: Oct. 25, 2025].
- [14] Red Hat, "ARM vs. x86: What's the difference?" *Red Hat*, Jul. 21, 2022. [Online]. Available: <https://www.redhat.com/en/topics/linux/ARM-vs-x86>. [Accessed: Oct. 25, 2025].
- [15] A. Serputov, "SVE2 in Armv9: What's New and Why It Matters," *dev.to*, 2021. [Online]. Available: <https://dev.to/serputov/sve2-Armv9-4o79>. [Accessed: Oct. 25, 2025].
- [16] ARM Ltd., "Armv9 CPUs: The Brains Behind Next-Gen Consumer Devices," *Arm Newsroom*, 2023. [Online]. Available: <https://newsroom.arm.com/blog/Armv9-cpus-consumer-devices>. [Accessed: Oct. 25, 2025].
- [17] S. Segars, "Armv9: The Future of Specialized Compute," *Arm Newsroom*, 2021. [Online]. Available: <https://newsroom.arm.com/blog/armv18>. [Accessed: Oct. 25, 2025].
- [18] "Arm's Solution to the Future Needs of AI, Security and Specialized Computing is v9," *Design & Reuse*, Mar. 30, 2021. [Online]. Available: <https://www.design-reuse.com/news/181761/arm-s-solution-to-the-future-needs-of-ai-security-and-specialized-computing-is-v18/>. [Accessed: Oct. 25, 2025].